SLO-02: Defining SLIs

> **Series:** SLO — Service Level Objectives | **Notebook:** 2 of 6 | **Created:** June 2026 | **Last Updated:** 08/31/2026

Overview

An SLI is a `good ÷ total` ratio expressed as a percentage. This notebook builds the four SLI shapes you will use most — availability, latency, error rate, and custom business SLIs — as DQL queries you can drop straight into a Dynatrace SLO. Every query here was validated against a live tenant.

The pattern is always the same: count the *good* events, count the *total* events, divide, multiply by 100. The skill is in defining "good" correctly for each indicator.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS Gen3 with Grail
Permissions	`storage:metrics:read`, `storage:spans:read`, `storage:bizevents:read` as applicable
Prior reading	SLO-01 (fundamentals)
MCP (optional)	Dynatrace MCP server to verify queries before adding them to an SLO

1. The SLI Ratio Pattern

Every SLI reduces to:

```
SLI % = (good events ÷ total events) × 100
```

In Dynatrace you express this as a DQL query whose result the SLO app samples over the evaluation window. Two query families serve different data:

- **Metric-based** (`timeseries`) — fast, pre-aggregated, ideal for service request metrics. Good and total are sums of metric series; divide element-wise with `[]`.
- **Event-based** (`fetch spans` / `logs` / `bizevents`) — flexible, lets you define "good" with any filter using `countIf()`. Higher query cost but expresses conditions a metric cannot.

Both are valid SLI sources. Reach for metric-based when a service metric already captures the condition; reach for event-based when "good" needs a predicate the metrics do not carry.

2. Availability SLI

Availability = the proportion of requests that did **not** fail. Using the pre-aggregated service request metrics, good = `total − failures`:

```
// Availability SLI — metric-based, request success ratio
// Re-validated on a live tenant 08/28/2026: returned 95.67%
// interval:24h, not 1d — a calendar duration here raises
// DEPRECATED_DURATION_UNIT_DAYS and is silently rewritten to 24h anyway
timeseries {
  total = sum(dt.service.request.count),
  failures = sum(dt.service.request.failure_count)
}, from:-30d, interval:24h
| fieldsAdd sli_availability = ((total[] - failures[]) / total[]) * 100
| fieldsAdd sli = arrayAvg(sli_availability)
| fields sli, total, failures
```

For an SLO, the SLI query is typically scoped to a specific service (add `, filter: {dt.entity.service == "SERVICE-..."}` or a tag filter) and the SLO app handles the windowing. The `sli` scalar is what the target is compared against.

3. Latency SLI — Two Approaches

Latency has no single canonical SLI in Dynatrace, because there is no built-in "count of requests faster than X" metric. There are two valid approaches — pick by what you actually need.

Decision

Approach	What it measures	Use when	Trade-off
Percentile threshold	Is the p95/p99 under a limit?	You care about the tail of the distribution	A threshold check, not a true good/total ratio — one number per window
Good/total ratio	What fraction of requests beat the limit?	You want a real SLI percentage and an error budget in "slow requests"	Computed from spans (event-based) — higher query cost

3a. Percentile-threshold (metric-based)

The p95 response time, in milliseconds (`dt.service.request.response_time` is in microseconds, so divide by 1000):

```
// Latency — p95 response time in ms (metric-based, validated)
// rollup goes INSIDE the function for percentile in timeseries
timeseries p95 = percentile(dt.service.request.response_time, 95, rollup:
avg),
  from:-24h, interval:1h
| fieldsAdd p95_ms = p95[] / 1000.0
| fieldsAdd p95_avg_ms = arrayAvg(p95_ms)
| fields p95_avg_ms
```

3b. Good/total ratio (span-based)

A true latency SLI: the fraction of server requests that completed under the threshold. `countIf()` defines "good" with a duration predicate:

```
// Latency SLI — span-based good/total ratio under 500ms
// Re-validated on a live tenant 08/28/2026: returned 99.54% (75,583/75,935)
// A 1h window on live traffic moves — treat the figure as a shape check,
// not a number to reproduce (06/2026 reading on the same tenant was ~96.4%)
fetch spans, from:-1h
| filter span.kind == "server"
| summarize {
    good = countIf(duration &lt; 500ms),
    total = count()
  }
| fieldsAdd sli_latency = (good * 100.0) / total
| fields sli_latency, good, total
```

> **Note the field values:** `span.kind` is lowercase (`"server"` , `"client"`), and `duration` compares directly against a duration literal (`500ms`) — no nanosecond constants. Both are common mistakes that silently return zero rows.

4. Error-Rate SLI

Error rate is availability's complement — sometimes you want to track it directly (and set the SLO as "error rate ≤ X%"). The good/total framing still applies; here "good" stays the success count, and you report the failure ratio alongside:

```
// Error-rate SLI — failure ratio, metric-based
timeseries {
  total = sum(dt.service.request.count),
  failures = sum(dt.service.request.failure_count)
}, from:-7d, interval:1h
| fieldsAdd error_rate = (failures[] / total[]) * 100
| fieldsAdd error_rate_avg = arrayAvg(error_rate)
| fields error_rate_avg
```

Express the SLO target as `success ≥ 99.5%` rather than `errors ≤ 0.5%` where you can — the SLO app and the error-budget math (SLO-03) are oriented around the "good" percentage, and keeping every SLI in the same direction avoids inverted-threshold confusion.

5. Custom / Business SLIs

The modern SLO app accepts any `good ÷ total` over any Grail data — which means business outcomes become first-class SLIs. Checkout success, order completion, or payment authorization rate computed from business events:

```
// Business SLI — checkout success rate from business events
// Adapt event.type / field names to your bizevent schema
fetch bizevents, from:-24h
| filter event.type == "checkout"
| summarize {
    good = countIf(checkout.result == "success"),
    total = count()
  }
| fieldsAdd sli_checkout = (good * 100.0) / total
| fields sli_checkout, good, total
```

Business SLIs are where SLOs earn executive attention — "99.9% of checkouts completed" lands in a way a service availability number does not. See the BIZEV series for building the business-event stream this query consumes.

6. Validate Before You Commit

An SLI query that returns nothing — wrong field name, wrong `span.kind` case, a filter that matches no data — produces an SLO that silently never computes. Before you save

an SLI into an SLO:

1. **Run it in a notebook** over a representative window and confirm it returns a sane percentage.
2. **Check the denominator.** A `total` of zero means the filter is too narrow; the SLI will be undefined.
3. **Verify with the MCP server** (`execute-dql`) against the live tenant if you have it wired up.
4. **Only then** paste it into the SLO definition.

This notebook-as-scratchpad discipline is the single best defence against a dashboard full of SLOs that read "no data."

> **Sources:** Service-Level Objectives (DT docs), DQL — timeseries and countIf (DT docs). All five queries re-executed against a live tenant 08/28/2026 (the bizevents example returns 0 rows there by design — it is a schema template, not a tenant claim). The availability query's `interval` was changed from `1d` to `24h` in the same pass: a calendar duration raises `DEPRECATED_DURATION_UNIT_DAYS` and is silently rewritten to `24h`, so the query was never running on the interval it appeared to specify.
